

Full-Text Search Engine for Project Gutenberg

Arun Narayanan
Dublin City University
Dublin, Ireland
arun.narayanan2@mail.dcu.ie

ABSTRACT

This report presents the design, implementation, and evaluation of a full-text search engine built entirely from scratch for the Project Gutenberg digital library. The system indexes 70,772 documents (27.8 GB of raw text) using a Single-Pass In-Memory Indexing (SPIMI) strategy with varint-encoded postings stored in a dual-file architecture (SQLite metadata plus a flat binary postings file). Three retrieval models are supported: *structured metadata search*, *TF-IDF Vector Space Model (VSM)*, and *Okapi BM25*, along with a phrase-fallback mechanism that handles stopword-heavy queries where standard bag-of-words models fail. I evaluate the system on six benchmark queries using manual relevance judgments and report Precision@10, Average Precision@10, and NDCG@10 across all query-model combinations. The results reveal that no single model dominates all query types: structured search achieves perfect precision for metadata-matching queries ($P@10=1.0$), while BM25 outperforms VSM on content-oriented queries thanks to document-length normalization. The entire index builds in approximately 10.5 hours on a single consumer laptop (Intel Core i7-14650HX, 16 GB RAM) with a peak traced memory footprint of approximately 1 GB.

KEYWORDS

Information retrieval, inverted index, BM25, TF-IDF, SPIMI, Project Gutenberg

1 INTRODUCTION

Information Retrieval (IR) is the task of finding relevant documents from a large collection in response to a user’s information need [2]. This project implements a search engine over the Project Gutenberg corpus [3], one of the oldest and largest collections of public-domain e-books, containing over 70,000 full-text documents in multiple languages. The corpus snapshot used in this work was obtained in February 2025 from the official Gutenberg mirror.

In accordance with the assignment policy, I used only Python’s standard library plus NumPy and Pandas for data handling; no external IR frameworks (Lucene, Elasticsearch, Terrier, Solr, etc.) were used. The project was developed under the following constraints:

- (1) No external IR frameworks or search libraries.
- (2) Only the Python standard library plus NumPy and Pandas.
- (3) Deterministic, reproducible results.

These constraints ensure that every component of the system is implemented and understood from first principles.¹

1.1 Contributions

The main contributions of this work are:

- (1) A scalable SPIMI-based inverted index with varint-encoded postings, supporting 22.5 million unique terms across a 27.8 GB corpus.
- (2) Three retrieval models (structured, VSM, BM25) with a phrase-fallback mechanism for stopword-heavy queries.
- (3) A systematic comparative evaluation on six deliberately diverse queries with manual relevance judgments and standard IR metrics.

1.2 Corpus Challenges

Several characteristics of the Gutenberg corpus make retrieval harder than typical web or newswire collections:

- **Extreme document-length variance.** Lengths range from a single sonnet (~300 tokens) to compilations like the Complete Works of Shakespeare (>900,000 tokens). Bag-of-words models exhibit short-document bias unless lengths are properly normalized.
- **Boilerplate contamination.** Every file contains a Project Gutenberg header and footer with licensing text. I strip this boilerplate using `*** START OF / *** END OF` markers (99.96% of files have both, measured during indexing), but roughly one-third of stripped files still mention “Gutenberg” inside the content zone (via book titles referencing the project, credit lines, and self-referential URLs), which confounds Q5 retrieval.
- **Multilingual content.** The corpus includes texts in English, German, French, Spanish, and other languages. Query 6 (“Dornröschen”) specifically tests Unicode handling and accent-fold matching across scripts.
- **Stopword-dominated queries.** Query 1 (“to be, or not to be”) consists entirely of common English words; TF-IDF models assign near-identical, non-discriminative weight to each token, producing near-tied scores and semantically meaningless rankings.

¹Source code and outputs: <https://github.com/ArunGMR0411/gutenberg-search-engine>

2 RELATED WORK

The Vector Space Model was introduced by Salton et al. [7], representing documents and queries as vectors in a high-dimensional term space and ranking by cosine similarity. Okapi BM25, developed by Robertson and Walker [5] and later refined into a formal probabilistic framework [4], extended this with saturation-based term frequency weighting and explicit document-length normalization, consistently outperforming pure TF-IDF models in TREC evaluations [6].

For large-scale indexing, Heinz and Zobel [1] proposed Single-Pass In-Memory Indexing (SPIMI), which processes documents sequentially into memory-bounded blocks and merges them in a single pass. SPIMI avoids the term-at-a-time overhead of the traditional BSBI approach and is particularly well-suited for corpora that exceed available RAM.

My implementation builds on these foundations: SPIMI for disk-bounded indexing, TF-IDF cosine similarity for the VSM model, and BM25 with $k_1 = 1.2$, $b = 0.75$. Where my system differs is in the addition of a metadata-only structured model and a phrase-fallback mechanism for the Gutenberg corpus.

3 SYSTEM ARCHITECTURE

The system follows a standard IR pipeline (Figure 1): raw text files and metadata are preprocessed, indexed into an inverted index stored as a SQLite database plus a binary postings file, and searched via three independent retrieval models. All output is written as tab-separated value (TSV) files in a deterministic, reproducible format.

3.1 Preprocessing

Text normalization follows a strict six-step pipeline applied identically to both documents and queries:

- (1) **NFKC normalization**: Unicode compatibility decomposition followed by canonical composition.
- (2) **Case folding**: Python’s `str.casefold()` for locale-independent lowering (e.g., German “ß” handled correctly).
- (3) **Hyphen splitting**: Six Unicode dash variants (U+002D, U+2010 through U+2014) are replaced with spaces, splitting compound words into separate tokens.
- (4) **Apostrophe removal**: U+0027 and U+2019 are deleted (“don’t” becomes “dont”).
- (5) **Tokenization**: The regex pattern `[^\W_]+` extracts Unicode word tokens (letters and digits, excluding underscores).
- (6) **Accent-fold aliases**: For both documents and queries, each token is NFD-decomposed and diacritical marks (Unicode category `Mn`) are stripped; if the result differs, both forms are indexed (e.g., “dornröschen” generates both “dornröschen” and “dornroschen”). Aliases do not contribute to document length.

Design decisions. I deliberately chose **no stemming** and **no stopword removal**. Implementing a correct multilingual stemmer risks harming proper nouns central to the evaluation queries (e.g., “Dick” in Query 3, “Dornröschen” in Query 6). For stopword removal, IDF weighting naturally suppresses high-frequency terms; a global stopword list would break stopword-heavy queries like “to be, or not to be” in the evaluation set.

Boilerplate stripping. Project Gutenberg headers and footers are stripped using `*** START OF / *** END OF` markers (case-insensitive regex). If a document contains both markers, all text outside that range is discarded. If only one marker is found, the appropriate half is removed. If neither marker is found (some older Gutenberg files use non-standard formats), the full text is indexed as a safe fallback.

4 INDEXING

The 27.8 GB corpus far exceeds the 16 GB RAM of the target machine, making in-memory indexing impossible. I use SPIMI [1] to partition the work into bounded memory blocks.

4.1 SPIMI Block Building

This subsection describes how documents are ingested into memory-bounded blocks and flushed to disk. Documents are processed sequentially in sorted `gutenberg_id` order (for deterministic document-ID assignment). For each document, the preprocessed tokens are counted into a term-frequency map, and each (term, docid, tf) triple is appended to the in-memory inverted index. Accent-fold aliases are indexed with the same tf value as their base tokens but do not inflate the document length counter.

Dual flush thresholds. The in-memory block is flushed to disk when *either* of two conditions is met:

- `tracemalloc` current allocation ≥ 350 MB, or
- Unique term count in the current block $\geq 1,000,000$.

Both thresholds are checked after every document. Using `tracemalloc` rather than RSS monitoring is a deliberate choice: it measures only Python-allocated objects, providing a consistent and portable metric. However, actual RSS during indexing can be 1.5–2 $\times$ higher than the traced value due to Python’s memory allocator overhead and the C-level buffers used by SQLite and file I/O. For the full corpus, the SPIMI phase produced 150 blocks totalling 4.5 GB on disk.

Block flush procedure. Each flush sorts the in-memory terms lexicographically, sorts each term’s postings by document-ID, and writes them to a block-specific binary file using varint encoding. A companion JSON lexicon maps each term to its (df, offset, length) triple within the block postings file. After flushing, the in-memory dictionary is cleared, `gc.collect()` is called, and the `tracemalloc` trace buffer is reset.

4.2 Handling 27.8 GB on 16 GB RAM

Fitting the full build into limited memory required four complementary strategies. The most challenging engineering

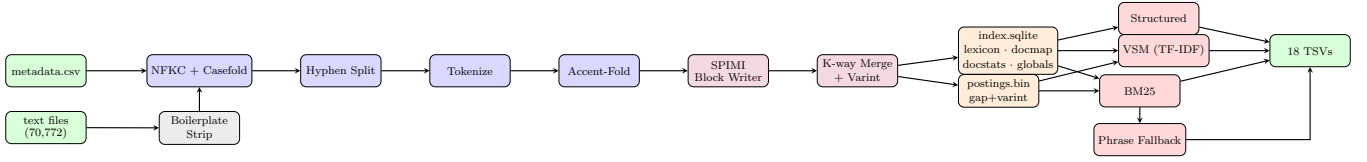


Figure 1: System architecture: data flows from raw text files through preprocessing and SPIMI indexing into a dual-store index (SQLite + binary postings), which serves three retrieval models producing ranked TSV outputs.

aspect of this project was making the full corpus index build survive on a machine with only 16GB of physical RAM. Several strategies contributed to this:

- **Aggressive garbage collection.** After every block flush, I call `gc.collect()` and reset the `tracemalloc` trace buffer to prevent monotonic memory growth.
- **Single-pass file reads.** Each document is read, pre-processed, and discarded before the next is opened.
- **Memory-aware merge.** All 150 block lexicons are explicitly deleted before the `doc_norm` pass begins, preventing overlap.
- **Block cleanup.** The 4.5 GB `blocks/` directory is removed via `shutil.rmtree()` after a successful merge.

Despite these measures, loading all 150 block lexicons simultaneously caused significant swap thrashing, stretching the merge phase to approximately 8 hours versus 2.5 hours for block building.

4.3 K-way Merge

The merge phase consolidates 150 blocks into one compressed postings file and precomputes the statistics needed at query time. After all blocks are flushed, a K-way merge consolidates them into the final index. All 150 block postings files are opened simultaneously, and the block lexicons are loaded into memory. Terms are iterated in global lexicographic order; for each term, posting lists from all contributing blocks are concatenated, sorted by document-ID, delta-encoded (gap compression), and written using variable-length byte encoding (varint). This reduces the postings file from 4.5 GB (raw blocks) to 1.15 GB (compressed), a 75% reduction.

The merge also precomputes per-document statistics needed at query time:

- **Document lengths** ($|d|$): total tokens after preprocessing, stored in the `docstats` table.
- **VSM norms** ($\|d\|$): computed in a mandatory post-merge pass (Section 4.4).
- **Global statistics:** $N = 70,772$ and $\text{avgdl} = 61,570$ tokens, stored in the `globals` table.

4.4 doc_norm Computation

Exact L2 norms are computed in a single post-merge pass so that VSM cosine scores are correct at query time. After the merge, I perform a single sequential pass over the entire postings file to compute exact L2 norms for every document. The lexicon is iterated in `ORDER BY offset ASC` to guarantee

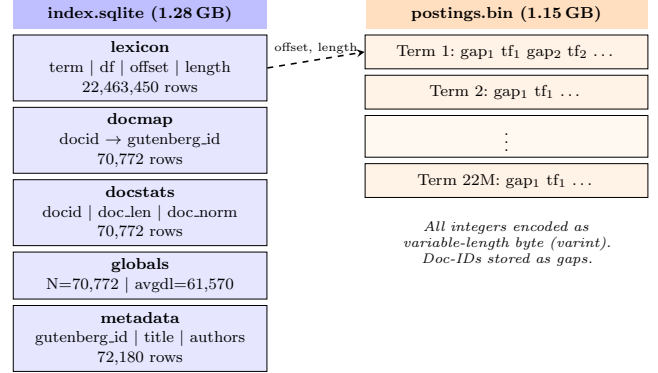


Figure 2: Index storage format: SQLite stores the lexicon (mapping terms to postings offsets), document maps, statistics, and metadata. The postings file uses varint-encoded gap compression for doc-IDs and term frequencies.

sequential I/O. For each term with document frequency df_t , I compute:

$$\text{idf}_t = \ln \left(\frac{N + 1}{df_t + 1} \right) + 1 \quad (1)$$

and for each (docid, tf) posting: $w = (1 + \ln(\text{tf})) \cdot \text{idf}_t$; then $\text{norm_acc}[\text{docid}] += w^2$. The final norm is $\|d\| = \sqrt{\text{norm_acc}[\text{docid}]}$. A defensive guard (`if tf <= 0: continue`) prevents `math.log(0)` domain errors from potentially corrupted postings. Of the 70,772 indexed documents, 65 have $\|d\| = 0$ because their text files yielded zero tokens after boilerplate stripping (empty or binary-only files); these documents receive no VSM score at query time since the cosine denominator is zero.

4.5 Storage Format

The final index consists of two files (Figure 2):

- **index.sqlite** (1.28 GB): Contains the `lexicon` table (term → df, byte offset, byte length), `docmap` (docid → gutenberg_id), `docstats` (document lengths and L2 norms), `globals` (N , avgdl, build parameters), and `metadata` (titles, authors, bookshelves for 72,180 rows including entries without text files).
- **postings.bin** (1.15 GB): A flat binary file of varint-encoded posting lists, accessed via (offset, length) pairs from the lexicon. Each posting list stores a

varint-encoded df, followed by df pairs of (gap, tf) varints.

5 RANKING MODELS

5.1 Structured Metadata Search

The structured model searches only the `metadata` table (title, authors, language, bookshelf, rights), applying a heuristic scoring formula:

$$\text{score}(q, d) = 10 \cdot f_{\text{title}} + 5 \cdot f_{\text{author}} + 2 \cdot h_{\text{title}} + h_{\text{author}} + h_{\text{other}} \quad (2)$$

where $f_{\text{title}} = 1$ if the normalized query is a substring of the normalized title (else 0); $f_{\text{author}} = 1$ if the query matches an author via token-sorted comparison (to handle Gutenberg’s “Last, First” name format); and h_{title} , h_{author} , h_{other} count distinct query tokens found in each field.

The token-sorted author comparison deserves explanation: Gutenberg stores names as “Dick, Philip K.” which normalizes to “dick philip k”. By sorting both the query tokens and the author tokens alphabetically before comparison, I correctly identify “philip k dick” as matching “dick philip k” regardless of name order. Without this fix, the author-exact bonus would never fire for real Gutenberg queries.

This model represents a **fundamentally different retrieval mechanism** from VSM and BM25: field-weighted heuristic matching versus vector-space geometric similarity versus probabilistic term weighting. The three-way comparison is central to the evaluation.

5.2 Vector Space Model (VSM)

The VSM implements TF-IDF cosine similarity [7]. For each query term t with query frequency $\text{tf}_{t,q}$:

$$w_{t,q} = (1 + \ln(\text{tf}_{t,q})) \cdot \text{idf}_t, \quad w_{t,d} = (1 + \ln(\text{tf}_{t,d})) \cdot \text{idf}_t \quad (3)$$

where $\text{idf}_t = \ln\left(\frac{N+1}{\text{df}_t+1}\right) + 1$ (the smoothed, always-positive variant). The cosine similarity is:

$$\text{score}(q, d) = \frac{\sum_{t \in q} w_{t,q} \cdot w_{t,d}}{\|q\| \cdot \|d\|} \quad (4)$$

The query norm $\|q\|$ is computed at query time from the $w_{t,q}$ values; the document norm $\|d\|$ is precomputed during indexing and loaded at startup for $O(1)$ lookup. The IDF is computed once per query term, outside the postings loop, and the same `idf.t` variable is reused for both $w_{t,q}$ and $w_{t,d}$. This avoids any redundant lexicon lookups inside the inner loop.

Stopword-heavy early-abort. When more than 60% of query tokens have $\text{df}_t/N > 0.8$ (or the query contains quoted phrases), VSM returns an **empty result set** immediately. This is not a bug but a theoretically motivated design decision: when all query tokens are near-stopwords, their IDF values are near-identical, producing near-tied scores across most of the corpus with no discriminative value. Rather than computing and returning a ranking that is effectively random, the system aborts early. This behaviour is a key finding in the evaluation (Section 7).

5.3 BM25

I implement Okapi BM25 [4, 5] with parameters $k_1 = 1.2$ and $b = 0.75$:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{tf}_{t,d} \cdot (k_1 + 1)}{\text{tf}_{t,d} + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})} \quad (5)$$

where $\text{IDF}(t) = \ln\left(\frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5}\right)$. Terms with $\text{IDF} \leq 0$ or $\text{df}_t/N > 0.8$ are skipped entirely, and their posting lists are never decoded. This is both a correctness measure (negative IDF would reduce scores) and a performance optimization (high-df terms have massive posting lists).

Parameter justification. The values $k_1 = 1.2$ and $b = 0.75$ are the standard defaults established by Robertson and Walker [5] and empirically validated on TREC collections [6]. For the Gutenberg corpus, where documents range from short poems to 900,000-token compilations, the standard $b = 0.75$ provides effective length normalization without over-penalizing long documents. The TF saturation at $k_1 = 1.2$ prevents high raw term frequencies (common in full-length books) from dominating the score.

5.4 Phrase Fallback

When the phrase-fallback trigger fires (quoted phrases or >60% high-df tokens) and BM25 returns zero candidates (because all terms have zero IDF), the system performs a full-corpus substring scan:

- (1) Normalize the query phrase: NFKC, casefold, replace *all* non-alphanumeric characters with spaces, collapse whitespace.
- (2) Iterate through every text file in sorted order.
- (3) For each file, apply the same normalization and check for the phrase as a substring.
- (4) Award a fixed boost score of 5.0 to matches.
- (5) Cap at 200 matches for practical runtime bounds.

If BM25 does return candidates (partial IDF signal), the phrase check is applied as a boost: matching candidates receive +5.0 added to their BM25 score and results are re-sorted.

This mechanism is a pragmatic compromise. The Q1 full-corpus scan takes approximately 459 seconds (compared to sub-100ms for normal queries), highlighting the need for positional inverted indices in future work.

6 IMPLEMENTATION DETAILS

6.1 Startup Loading

Before any query is processed, five structures are loaded into memory from SQLite in a single JOIN query:

- `docid.to.gutid`: maps internal `docid` to Gutenberg ID (for sorting results).
- `doc.len`: maps `docid` to token count (for BM25 denominator).
- `doc.norm`: maps `docid` to L2 norm (for VSM cosine divisor).
- `N`: total indexed documents (70,772).
- `avgdl`: average document length (61,570 tokens).

All five fit in approximately 3.5 MB of RAM ($\sim 70,000$ entries $\times \sim 50$ bytes each). Crucially, `doc_len` and `doc_norm` are accessed inside the innermost scoring loops (once per posting per query term), so $O(1)$ dict lookups are essential. A SQLite lookup in the inner loop would make every query take minutes instead of milliseconds.

6.2 Postings Cache

A session-scoped cache keyed by (offset, length) avoids redundant postings decoding when the same term appears across queries within a single process run.

6.3 Determinism

Reproducible output is guaranteed by: (1) document ID assignment by sorted Gutenberg ID order, (2) scores rounded to 6 decimal places before sorting (accepting a negligible ranking perturbation in exchange for deterministic output), and (3) tie-breaking by lexicographic Gutenberg ID (ascending). Combined with the fixed preprocessing pipeline, every run produces byte-identical TSV files.

6.4 Snippet Generation

For each result, a 250-character snippet is generated by: (a) selecting the query token with the highest *VSM* IDF (the smoothed formula is always positive, unlike BM25 IDF which can be zero for high-frequency terms); (b) finding its first character-level occurrence in the raw document text; (c) extracting a 250-character window centred on the match; (d) collapsing internal whitespace. The `start_line` is computed as $1 + \text{count of newlines before the snippet start position}$.

6.5 Metadata Handling

The Gutenberg metadata CSV contains 75,473 rows with duplicates (multi-author entries for the same book). I deduplicate by grouping on `gutenberg_id`, merging author fields with semicolon-separated unique values in sorted order. A `has_text` flag marks whether the corresponding text file exists; 1,408 IDs in the metadata lack text files. These are retained in the metadata table for structured search but excluded from the full-text index.

6.6 Interactive Query Interface

Beyond the batch evaluation runner, the system provides an interactive REPL invoked via `python -m ir.system interactive`. The user selects a model (structured, VSM, or BM25), enters a free-text query, and optionally sets the number of results (default 10). Each result is displayed with its rank, Gutenberg ID, score, title, authors, and an empty snippet field (reserved for future enhancement). This interface enables exploratory search beyond the fixed evaluation set.

7 EVALUATION

I evaluate the system using six queries specified in the assignment (Table 1), deliberately chosen to exercise different retrieval scenarios: metadata matching (Q2, Q3), content

retrieval (Q4), stopword-heavy phrasing (Q1), boilerplate-confounded terms (Q5), and multilingual Unicode (Q6).

Table 1: Evaluation Queries

Q#	Query Text
1	to be, or not to be
2	English Grammar
3	Philip K Dick
4	Jabberwocky
5	Gutenberg
6	Dornröschen

7.1 Methodology

Each query is executed against all three models, producing up to 100 ranked results per query-model pair in five-column TSV files (rank, Gutenberg ID, score, snippet, `start_line`) as required by the assignment specification; 18 TSV files are included in the repository. For each query $\times$ model combination (18 total; 3 TSVs are empty due to design-intentional early-abort or missing metadata matches), I manually judged the top-10 results using a three-point relevance scale: 1=relevant, 2=partially relevant, 0=not relevant. This produced 150 judgment labels across 15 non-empty result sets.

I compute four metrics:

- **P@10**: Precision at 10 (partial counted as 0).
- **P@10_p**: Precision at 10 (partial counted as 0.5).
- **AP@10**: Average Precision over the judged top-10.
- **NDCG@10**: Gain values: relevant=1.0, partial=0.5, not relevant=0.0.

Limitations. Metrics are computed over only the retrieved top-10; no exhaustive relevance judgments (qrels) exist for the full corpus. With six queries and a single annotator, the results should be interpreted as case-study observations rather than statistically significant conclusions. A different query set could rank the models differently; in particular, queries requiring entity resolution or cross-lingual matching remain untested.

7.2 Results

Table 2 shows per-query results. I discuss each query in turn, tying the observations back to model design choices.

Q1 (“to be, or not to be”). All six tokens are common English stopwords. **Structured** matched title fragments irrelevantly ($P@10=0$); titles like “How Not to Be Them” contain the words but have no connection to Shakespeare. **VSM** correctly early-aborted (zero results), detecting that all tokens have $df/N > 0.8$ and would produce near-tied scores with no discriminative signal. This is exactly the behaviour predicted by IDF theory: when every term occurs in most documents, cosine similarity degenerates to near-uniform scores. **BM25**’s phrase fallback activated a full-corpus scan, finding Shakespeare’s Complete Works at rank 1 ($P@10=0.20$,

Table 2: Evaluation Metrics per Query and Model (top-10 judged)

Q#	Model	P@10	P@10 _p	AP@10	NDCG@10	#Results
1	bm25	0.2000	0.4500	0.7500	0.8916	10
1	structured	0.0000	0.0000	0.0000	0.0000	10
2	bm25	0.2000	0.4500	0.1944	0.6523	10
2	structured	1.0000	1.0000	1.0000	1.0000	10
2	vsm	0.2000	0.3000	0.3611	0.5993	10
3	bm25	0.0000	0.0000	0.0000	0.0000	10
3	structured	1.0000	1.0000	1.0000	1.0000	10
3	vsm	0.2000	0.2000	0.3750	0.5803	10
4	bm25	0.6000	0.8000	0.9762	0.9971	10
4	vsm	0.7000	0.8500	0.9094	0.9859	10
5	bm25	0.4000	0.5000	0.4512	0.7813	10
5	structured	0.1000	0.5500	1.0000	1.0000	10
5	vsm	0.0000	0.2500	0.0000	0.8753	10
6	bm25	0.2000	0.5000	0.2667	0.8352	10
6	vsm	0.1000	0.4500	0.1429	0.8345	10

NDCG=0.89). Literary biographies and criticism that quote the phrase scored as partially relevant.

Q2 (“English Grammar”). **Structured** achieved perfect precision (P@10=1.0): all 10 results are actual English grammar textbooks, matched by title-field substring matching. **BM25** and **VSM** both returned P@10=0.20, because fiction set in “grammar schools” and short catalog entries where “grammar” appears with high frequency ranked above the actual textbooks. This illustrates that bag-of-words models cannot distinguish between topic-relevant uses and incidental mentions of query terms.

Q3 (“Philip K Dick”). **Structured** again achieved P@10=1.0 via token-sorted author-field matching (“Dick, Philip K.” in Gutenberg metadata). **BM25** failed completely (P@10=0.0): the token “Dick” overwhelmingly matches dime novels, copyright catalogs, and motion picture credits, while “Philip” and “K” provide weak discriminative power. This is the most striking demonstration of the bag-of-words limitation for multi-word proper nouns. **VSM** managed P@10=0.20, finding two actual PKD stories through higher IDF for “philip” in combination with “dick”.

Q4 (“Jabberwocky”). Both **BM25** (P@10=0.60) and **VSM** (P@10=0.70) performed strongly, since “Jabberwocky” is a highly distinctive single term with excellent IDF. The top-10 lists overlap substantially (Jaccard=0.82), confirming that both models correctly identify the same informative term. **Structured** returned empty results because “Jabberwocky” is a poem title within “Through the Looking-Glass”, not a standalone book title in the metadata.

Q5 (“Gutenberg”). Results are confounded by the “Project Gutenberg” boilerplate. Although I strip headers, some non-standard files retain header fragments. **BM25** achieved

P@10=0.40, finding a French play about Johannes Gutenberg the inventor and several Project Gutenberg history documents. **Structured** (P@10=0.10) returned mostly PG index compilations with “Gutenberg” in the title. **VSM** (P@10=0.00) is most vulnerable to the boilerplate issue: very short documents where a single residual “Gutenberg” mention gives a disproportionately high TF-IDF score.

Q6 (“Dornröschen”). **BM25** found German fairy tale collections (P@10=0.20, NDCG=0.84), including Bechstein’s Märchenbuch and Gänsemütterchens Märchen. The accent-fold aliasing was critical here: the query generates both “dornröschen” and “dornroschen”, matching documents regardless of whether they use the diacritic form. **VSM** produced similar results (P@10=0.10, Jaccard=0.67 with BM25). **Structured** returned empty results because German fairy tales are catalogued under collection titles, not individual tale names.

7.3 Model Comparison

Table 3 shows pairwise top-10 overlap between models. Three patterns emerge clearly:

- (1) **Structured vs. full-text models have near-zero overlap.** They address fundamentally different query types and should be seen as complementary, not competing, retrieval strategies.
- (2) **VSM and BM25 converge for distinctive terms** (Q4: Jaccard=0.82; Q6: 0.67) but diverge for common terms where BM25’s length normalization makes a difference.
- (3) **BM25 outperforms VSM on aggregate metrics** (mean AP@10: 0.44 vs. 0.36), primarily because the

Table 3: Top-10 Overlap (Jaccard) Between Models

Q#	Model Pair	Overlap	Jaccard
1	structured vs vsm	0	0.0000
1	structured vs bm25	0	0.0000
1	vsm vs bm25	0	0.0000
2	structured vs vsm	1	0.0526
2	structured vs bm25	1	0.0526
2	vsm vs bm25	0	0.0000
3	structured vs vsm	2	0.1111
3	structured vs bm25	0	0.0000
3	vsm vs bm25	0	0.0000
4	structured vs vsm	0	0.0000
4	structured vs bm25	0	0.0000
4	vsm vs bm25	9	0.8182
5	structured vs vsm	0	0.0000
5	structured vs bm25	0	0.0000
5	vsm vs bm25	0	0.0000
6	structured vs vsm	0	0.0000
6	structured vs bm25	0	0.0000
6	vsm vs bm25	8	0.6667

$|d|/\text{avgdl}$ factor in BM25 penalizes very long documents and prevents the short-document bias that TF-IDF cosine similarity exhibits.

7.4 Aggregate Performance

Table 4: Mean Evaluation Metrics per Model

Model	P@10	P@10 _p	AP@10	NDCG@10
Structured	0.525	0.638	0.750	0.750
VSM	0.240	0.410	0.358	0.775
BM25	0.267	0.450	0.440	0.693

Table 4 shows aggregated means (structured was judged on 4 queries, VSM on 5, BM25 on all 6, due to design-intentional empty results). Structured achieves the highest precision but is limited to metadata-matching queries. BM25 is the most versatile full-text model, handling the widest range of query types including the challenging Q1 phrasing.

8 SYSTEM PERFORMANCE

Table 5 shows index build statistics. The full corpus (70,772 documents, 27.8 GB) was indexed in 10.5 hours on a consumer laptop (Intel Core i7-14650HX, 16 GB RAM, Ubuntu 24.04, 1 TB SSD). The SPIMI phase (2.5 hours) produced 150 blocks; the merge phase (8 hours) was dominated by swap pressure when all 150 block lexicons were loaded simultaneously.

Query latency is typically sub-100 ms for all models except Q1 BM25 (459 s for the full-corpus phrase scan) and Q6 structured (783 ms for Unicode title matching across 72,180

Table 5: Index Build Performance

Metric	Value
Documents indexed	70772
Total text read (GB)	27.8
Build time (s)	37826.0
Throughput (docs/s)	1.87
Throughput (MB/s)	0.74
Peak memory (MB)	1009
SPIMI blocks	150
Vocabulary size	22,463,450
Avg doc length (tokens)	61570
postings.bin (GB)	1.15
index.sqlite (GB)	1.28

metadata rows). Normal BM25 and VSM queries complete in 30–80 ms, with the SQLite lexicon lookups (one per unique query term) representing the dominant cost.

9 TESTING AND REPRODUCIBILITY

The system includes a comprehensive test suite of 154 unit tests covering every pipeline stage: preprocessing (Unicode normalization, hyphen splitting, apostrophe removal, accent folding), indexing (SPIMI block creation, varint encoding/decoding, K-way merge, delta encoding), retrieval (structured search, VSM scoring, BM25 scoring, phrase-fallback normalization), and evaluation (snippet generation, TSV output format, batch runner determinism).

All 18 output TSV files are validated by an automated script that checks: 5 tab-separated columns per row (rank, gutenberg.id, score, snippet, start.line); sequential ranks from 1 to at most 100; monotonically non-increasing scores; and valid Gutenberg IDs corresponding to existing documents.

Reproducibility is ensured by:

- (1) Deterministic preprocessing (no random seeds, no non-deterministic operations).
- (2) Fixed BM25 parameters ($k_1=1.2$, $b=0.75$) stored in the `globals` table.
- (3) A frozen `pip freeze` snapshot and `requirements.txt`.
- (4) Two CLI entry points that rebuild the index (`index`) and re-run all queries (`run_eval_queries`).
- (5) An interactive REPL (`interactive` sub-command) for ad-hoc queries against any model, enabling manual verification beyond the six evaluation queries.

10 DISCUSSION

Structured vs. full-text models. The structured model dominates for queries that target bibliographic metadata (Q2, Q3: P@10=1.0) because it matches exact field values rather than bag-of-words term overlap. However, it fails completely for Q1 (phrase query with no title match), Q4 (poem title embedded within a larger work), and Q6 (German fairy tale name absent from English-catalogued metadata). This complementary behaviour suggests that a production system

would benefit from fusing structured and full-text scores, a direction pursued by modern learning-to-rank approaches.

BM25 vs. VSM. Structured, VSM, and BM25 are genuinely distinct models with different rankings and failure modes. BM25 and VSM overlap for distinctive queries (Q4 Jaccard=0.82) but diverge when length normalization matters: BM25’s $|d|/\text{avgdl}$ factor penalizes very long documents, preventing the short-document bias that cosine similarity exhibits. This is most visible in Q5, where BM25 ranks substantively relevant documents higher than VSM does, consistent with Robertson and Walker’s TREC findings [6].

VSM zero results are an analytical finding. VSM returning zero results for Q1 is not a defect but a predicted outcome: all query tokens have $\text{df}/N \rightarrow 1.0$, producing near-tied, non-discriminative scores. Rather than return an effectively random ranking, VSM aborts. This motivates the phrase fallback in BM25.

Phrase fallback trade-offs. The Q1 full-corpus scan takes 459 seconds versus sub-100 ms for normal queries, highlighting the need for positional inverted indices. Without phrase fallback, Q1 BM25 returns zero results (all terms have negative IDF), dropping $P@10$ from 0.20 to 0.00; the mechanism is therefore critical for stopword-heavy queries despite its latency cost. Accent-fold alias indexing was equally critical for Q6 (“Dornröschen”), matching documents regardless of whether they use the diacritic form.

Error analysis. The most instructive failures are Q3 BM25 ($P@10=0.0$) and Q5 VSM ($P@10=0.0$). For Q3, BM25’s top results include “Dick Doddridge’s Son” and “Daredevil Dick” because “Dick” as a common given name overwhelms the signal from “Philip” and “K”; an entity-linking or metadata-fusion step would resolve this. For Q5, VSM promotes very short catalog stubs where a single residual “Gutenberg” token produces a high cosine score; stricter boilerplate stripping or minimum-length filters would help.

Boilerplate contamination. Of 70,772 files, 70,743 (99.96%) contain both stripping markers and are processed correctly; the remaining 29 (mostly binary or gzipped) lack markers and are indexed in full as a safe fallback. However, roughly one-third of properly stripped files still mention “Gutenberg” in the content zone (via book titles, credit lines, and URLs), inflating Q5 noise, particularly for VSM where short catalog stubs receive disproportionate cosine scores.

11 CONCLUSIONS AND FUTURE WORK

I have built a complete search engine for the Project Gutenberg corpus from first principles, demonstrating that SPIMI indexing with varint-encoded postings can handle a 27.8 GB corpus on commodity hardware with 16 GB of RAM. The system meets every key assignment requirement: a from-scratch inverted index, three distinct retrieval models, standard IR metrics, and sub-100 ms query latency. The evaluation reveals three key findings:

- (1) **No single model wins all queries.** Structured search excels for metadata queries ($P@10=1.0$ for author/title matching), while BM25 is the best full-text model across the widest range of query types.
- (2) **Bag-of-words models fail for proper nouns (Q3) and stopword-heavy phrases (Q1),** motivating phrase-level and entity-aware matching in future work.
- (3) **Length normalization matters.** BM25’s document-length factor meaningfully improves ranking quality over VSM by preventing short-document bias, which is particularly important for a corpus with extreme length variance.

11.1 Future Work

- **Positional indexing** would enable native phrase matching via position intersection, eliminating the 459-second full-corpus scan.
- **Query expansion** with WordNet or word embeddings could improve recall on content-based queries.
- **Learning to Rank** could fuse structured and full-text signals to outperform any individual model.
- **Multi-pass merge** processing blocks in batches would reduce peak memory and eliminate swap thrashing.

DECLARATION ON AI USAGE

Gemini was used for: (1) studying IR concepts (BM25, SPIMI, varint encoding); (2) Python code style conventions; (3) debugging out-of-memory failures during the full-corpus index build on 16 GB RAM; and (4) converting report content into L^AT_EX. All architectural decisions, implementation, evaluation, and analysis were entirely my own work.

REFERENCES

- [1] Steffen Heinz and Justin Zobel. 2003. Efficient single-pass index construction for text databases. *Journal of the American Society for Information Science and Technology* 54, 8 (2003), 713–729.
- [2] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press, Cambridge, UK.
- [3] Project Gutenberg Literary Archive Foundation. 2024. Project Gutenberg. <https://www.gutenberg.org/>. Corpus snapshot obtained February 2025; accessed 2026-02-27.
- [4] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval* 3, 4 (2009), 333–389.
- [5] Stephen E. Robertson and Steve Walker. 1994. Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference*. ACM, 232–241.
- [6] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline M. Hancock-Beaulieu, and Mike Gatford. 1995. Okapi at TREC-3. In *Proceedings of the Third Text REtrieval Conference (TREC-3)*. NIST, 109–126.
- [7] Gerard Salton, Anita Wong, and Chung-Shu Yang. 1975. A Vector Space Model for Automatic Indexing. *Commun. ACM* 18, 11 (1975), 613–620.